

Computation-Aware Caching for KV Cache

Yiyu Liu
Harvard University
yiyuliu@g.harvard.edu

Abstract

As Large Language Models (LLMs) scale to handle massive context windows, the computational cost of the Key-Value (KV) cache becomes a dominant factor in serving performance. Existing prefix caching strategies often ignore the quadratic complexity of self-attention, treating all cache blocks as equally valuable despite the vast recomputation asymmetry between root and leaf tokens. In this report, we present CHAFRA (Cache-Hole-Aware Frequency-Recency Algorithm), a system that optimizes prefix caching by accounting for both computational costs and cache fragmentation. We introduce CAFRA (Cache-Aware Frequency-Recency Algorithm), an online eviction policy that prioritizes deep, computationally expensive blocks. Additionally, CHAFRA implements a hole-aware management system that enables the reuse of fragmented cached segments through batched hole filling. Our evaluation on production traces shows that CHAFRA significantly reduces Time-to-First-Token (TTFT) by protecting high-value blocks, demonstrating that computation-aware caching is essential for efficient long-context LLM serving.

1 Introduction

The deployment of Large Language Models (LLMs) in production environments is increasingly constrained by the computational and memory overhead of the Key-Value (KV) cache. As applications evolve from simple queries to long-context tasks such as document analysis and autonomous agents, sequence lengths have grown from a few thousand to hundreds of thousands of tokens. To mitigate the resulting quadratic growth in attention computation, modern serving engines utilize prefix caching to reuse KV states across requests.

However, existing prefix caching systems often treat all cache blocks as equal in value. This approach ignores a fundamental property of the Transformer architecture: because of causal masking and the $O(N^2)$ complexity of attention, recomputing a token at a 100K-token depth is orders of magnitude

more expensive than recomputing a token at the root of a prefix tree. Consequently, structure-agnostic eviction policies like Least Recently Used (LRU) can lead to severe latency spikes by evicting “deep” blocks that are extremely costly to regenerate. Furthermore, block-level eviction natively creates fragmentation or “holes” in the cached sequence, which can degrade prefill performance if not managed carefully.

In this final report, we present CHAFRA (Cache-Hole-Aware Frequency-Recency Algorithm), a system that optimizes prefix caching by accounting for both computational asymmetry and cache fragmentation. We introduce CAFRA (Cache-Aware Frequency-Recency Algorithm), an online eviction algorithm that prioritizes retaining expensive-to-compute blocks by integrating frequency, recency, and recomputation cost into a unified metric. To handle fragmentation, we design a hole-aware system that enables the reuse of non-contiguous cached segments through an optimized batched hole-filling mechanism.

We implement CHAFRA within the vLLM serving engine and evaluate our approach against standard baseline policies. Our results demonstrate that by making the cache computation-aware, we can significantly reduce the Time-to-First-Token (TTFT) and improve overall system throughput in long-context workloads. The remainder of this report details the motivation for our design, the formalization of our algorithms, and the system-level implementation details of our hole-aware architecture.

2 Background

2.1 Key-Value (KV) Cache

To accelerate the autoregressive generation process in Transformer-based models, serving engines utilize a Key-Value (KV) cache. During inference, each token’s generation requires computing attention over all previously processed tokens. Specifically, the model requires the Key (K) and Value (V) tensors for all tokens in the context. Rather than recomputing these tensors from scratch for every new

token, the system stores the intermediate K and V activations in memory—typically GPU HBM—once they are computed. This allows the decoding phase to perform incremental attention by only computing the K and V vectors for the current token and retrieving the rest from the cache. However, the KV cache consumes significant memory, scaling linearly with the sequence length, batch size, and model width [13]. In long-context scenarios, the KV cache size can exceed tens of gigabytes, making efficient memory management and reuse critical for high-throughput serving [9].

2.2 The Quadratic Wall in Attention

Large Language Models (LLMs) based on the Transformer architecture [18] rely heavily on the self-attention mechanism to model token dependencies. However, exact self-attention requires calculating the dot products between all pairs of tokens, leading to an arithmetic complexity that scales quadratically with the sequence length ($O(N^2)$) [8]. During LLM inference, when the sequence length grows to support complex use cases (e.g., document question-answering [3, 10] and autonomous agents [12, 19]), the prefill phase—where the input prompt is processed to generate the initial Key-Value (KV) cache—becomes a severe computational bottleneck. The time required for this phase dominates the Time-to-First-Token (TTFT), directly impacting user experience.

2.3 Prefix Caching

To mitigate the overhead of repeated attention computation, modern LLM serving engines have adopted multi-request prefix caching (e.g., SGLang [20], vLLM [9]). In many real-world workloads, concurrent or sequential requests frequently share common prefixes, such as lengthy system prompts [12], few-shot examples [5], or prior history [19].

Crucially, the autoregressive nature of Transformer decoders relies on causal masking, meaning the attention state of a token depends strictly on itself and all preceding tokens. Consequently, the KV cache can only be reused if the entire preceding sequence exactly matches. This strict prefix dependency naturally structures the reusable KV cache as a prefix tree (or radix tree for better memory efficiency, as introduced by SGLang [20]).

Instead of recomputing the KV states from scratch for every request, the system caches the KV blocks of previously processed prefixes as nodes in the radix tree. These serving engines can efficiently match the common prefix of a new request by traversing the radix tree and reuse the cached KV states. This multi-request reuse bypasses redundant prefill computations, significantly reducing TTFT and increasing overall system throughput. However, because GPU memory is fundamentally limited, serving engines must eventually evict cached nodes to accommodate new requests.

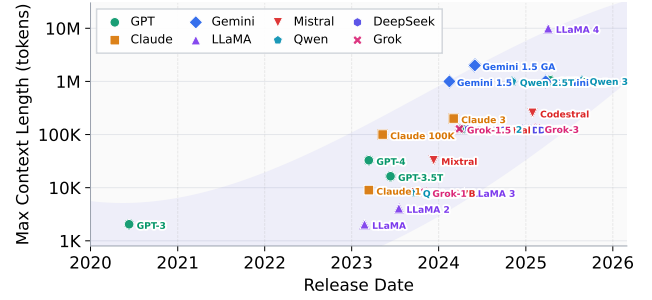


Figure 1: Evolution of maximum model context length across major model families (2020–2026). The shaded band represents the dynamic distribution of model capabilities, showing a rapid expansion in context windows since 2023.

2.4 The Decoupling of Tree Structure and Cache Eviction

While the prefix tree provides an effective logical data structure for organizing and matching shared KV states, its structural maintenance is fundamentally decoupled from physical cache eviction policies. The radix tree naturally enforces logical dependencies—a child node cannot exist without its parent—but it does not inherently dictate which specific nodes should be evicted when GPU memory is exhausted.

This decoupling means that physical eviction operates independently of the tree’s structural representation. When an eviction policy, such as Least Recently Used (LRU), removes a node to free memory, it must only ensure that the logical dependencies of the tree remain intact (e.g., by evicting from the leaf nodes upwards). However, because the structural dependency is separated from the physical eviction, the eviction policy focuses solely on maximizing cache hit rates based on temporal locality, without inherently accounting for the computational asymmetry of the prefix tree.

3 Motivation

3.1 The Necessity of Prefix Caching

Modern LLM applications have evolved significantly from simple single-turn queries. Conversational chatbots now maintain extensive multi-turn histories to provide contextual and personalized responses, while autonomous system agents operate with massive system prompts containing detailed instructions, available tool descriptions, and complex reasoning scratchpads. This evolution has driven the demand for ever-increasing context windows (as shown in Figure 1). For instance, a recent study by OpenRouter [2] reveals that the average sequence length of LLM requests has more than tripled, growing from under 2,000 tokens in late 2023 to over 5,400 tokens by late 2025. The same study shows that this trend is even more pronounced for programming work-

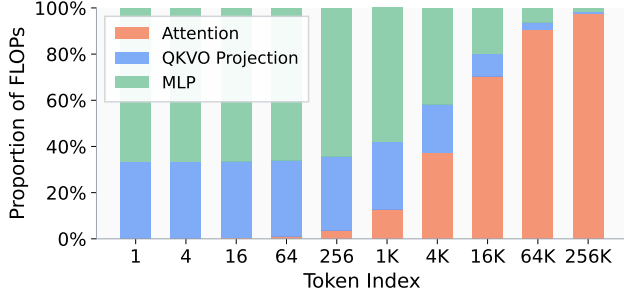


Figure 2: Proportion of per-token FLOPs dedicated to Attention, MLP, and QKVO projections during the prefill phase for Qwen3 Coder 30B model [17]. As the sequence length grows, attention rapidly dominates the computational cost.

flows, where sequences routinely exceed 20,000 tokens. Consequently, the length of prefill requests has grown substantially, making the prompt processing phase a dominant factor in latency. Because these long contexts frequently share identical prefixes—such as the same grounding documents or system instructions across multiple queries—prefix caching is highly effective. In fact, production traces show that the cache hit ratio remains consistently high. Experimental evaluations of Mooncake’s [15] disaggregated KV Cache show it can theoretically process up to 75% of tokens from cache, with actual block-level hit ratios routinely exceeding 50% and sometimes 80% in optimal conditions. More broadly, production deployments that leverage prefix reuse, such as Red Hat’s llm-d [7], Character.AI [6], and agentic frameworks like Manus [14], report cache hit rates frequently exceeding 90% or even 95%—reinforcing its necessity.

3.2 The Shift: MoE and the “Attention Pressure”

Simultaneous to the growth in context length, model architectures have shifted toward Mixture-of-Experts (MoE) designs (e.g., Mixtral, Qwen-MoE) [16, 21]. In dense Transformer models, the Multi-Layer Perceptron (MLP) layers dominate the parameter count and computation time during the prefill phase. However, MoE architectures substantially accelerate the MLP stages by activating only a sparse subset of experts for each token.

Because the attention mechanism must still calculate dense interactions across all tokens in the sequence, the relative cost of attention computation increases dramatically compared to the MLP projections. Figure 2 illustrates this shifting bottleneck for a modern MoE model (Qwen3-Coder-30B-A3B-Instruct [17]). While MLP and QKVO projections require a constant number of operations per token, the exact self-attention computation scales linearly with the token’s position index. For deep tokens (e.g., near the 200K limit), the attention mechanism absorbs the vast majority of the floating-point operations. As a result, long-context LLM serving faces

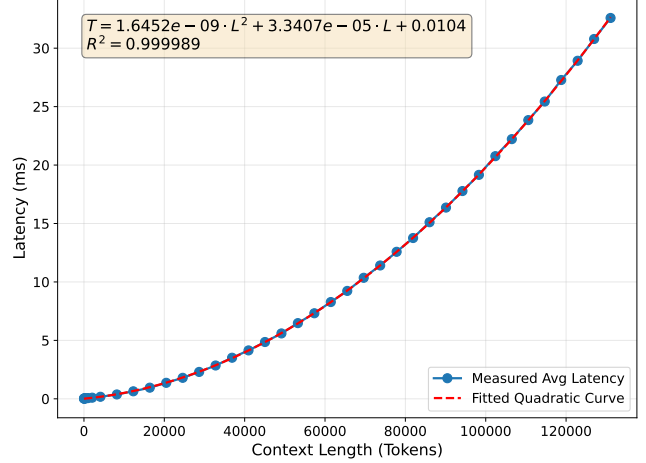


Figure 3: Time-To-First-Token (TTFT) as sequence length increases for a full prefill without prefix cache hits on the Qwen3-Coder-30B model. As shown by the nearly perfect quadratic fit, the execution time scales quadratically rather than linearly, demonstrating the severe latency impact of processing deep tokens and motivating computation-aware eviction.

immense “attention pressure”: the computational bottleneck of the prefill phase shifts heavily toward the attention layers.

3.3 The Necessity of Computation-Aware Eviction

Because of the quadratic arithmetic complexity of exact self-attention ($O(N^2)$), the computational cost of processing a token is strictly dependent on its position in the sequence. As shown by the nearly perfect quadratic fit ($R^2 = 0.999989$) in Figure 3, the Time-To-First-Token (TTFT) for a full prefill grows quadratically—not linearly—with sequence length due to this attention overhead. Recomputing a token deep in the prefix tree (e.g., at index 100,000) requires calculating dot products with all 99,999 preceding tokens. Conversely, recomputing a token near the root of the tree requires negligible attention overhead. Therefore, the blocks at different indices have vastly different recomputation times.

When a structure-agnostic algorithm like LRU treats all cache blocks equally, it often evicts deep, highly expensive blocks simply because they were least recently used. The next request that relies on that evicted block will suffer a massive TTFT spike due to the $O(N^2)$ recomputation penalty. To prevent this, a computation-aware eviction strategy is critical. By ensuring that blocks requiring the most compute to regenerate are protected, the system shifts its optimization objective from simply maximizing naive *cache hit rates* to minimizing *overall computational overhead*, thereby protecting user experience.

For example, consider the cost of recomputing a sequence of 1,024 tokens using the Qwen3-Coder-30B-A3B-Instruct

model configuration (from Figure 2). If the system must recompute 1,024 tokens at the root of the prefix tree (indices 0 to 1,023), the total computational cost is approximately 0.13 TFLOPs. However, if those same 1,024 tokens are located deep in the context (starting at index 65,536 (64K)), the required operations skyrocket to 1.22 TFLOPs. Because the exact self-attention overhead scales linearly with each token’s position, recomputing a block at a 64K depth is nearly $10\times$ more computationally expensive than recomputing an equivalent block at the root. Empirically, this translates directly to severe latency spikes: prefilling a 1K-token block at the root yields a Time-To-First-Token (TTFT) of only 58 ms, whereas prefilling the exact same 1K block after a 64K cached prefix takes 335 ms—a nearly $6\times$ latency degradation.

4 Cache Algorithm Design

Here, we focus on improving the efficiency of the prefix cache algorithm in terms of computation. In §2.4, we have already discussed that the tree structure actually have nothing to do with the cache eviction, i.e., we can evict not only the leaf nodes, but also the internal nodes. In this section, we will start with an pessimistic offline optimal policy, followed by an efficient offline algorithm, and finally introduce online algorithms.

4.1 The Pessimistic Offline Optimal Policy

To dig into the potential of the prefix cache algorithm, we first introduce our Pessimistic Offline Optimal Policy. The pessimistic offline optimal policy can be represented by an integer linear programming (ILP) problem.

Let I be the set of all reuse intervals across the workload. A reuse interval $i \in I$ is defined as the temporal span between two consecutive accesses to the same block. By formulating the variables as reuse intervals, blocks accessed only once (i.e., one-hit wonders) do not form a closed interval and are naturally excluded from the problem. For each reuse interval $i \in I$, let c_i denote its cost (e.g., the computation saved if the block is cached) and let s_i be the size of the block. We introduce a binary decision variable $x_i \in \{0, 1\}$, where $x_i = 1$ if the block is kept in the cache for the entire duration of interval i , and $x_i = 0$ otherwise.

The objective is to maximize the total cost of the cached reuse intervals:

$$\max \sum_{i \in I} c_i x_i. \quad (1)$$

Traditionally, a cache must enforce a capacity check at every fine-grained execution step. However, this introduces an excessive number of constraints. To ensure space availability while keeping the formulation computationally tractable, our policy evaluates the capacity constraint *only before the first block of each request*.

Let $\mathcal{R}$ be the set of all requests, where each request $r \in \mathcal{R}$ starts at time t_r and has a length (or size) of L_r . Let $I_r \subseteq I$ be the set of reuse intervals that are active precisely at time t_r (i.e., the interval spans across the arrival of the request, $\text{start}_i \leq t_r < \text{end}_i$). The cache capacity constraint dictates that right before executing the request, the sum of all cached blocks plus the length of the incoming request must not exceed the cache capacity:

$$\sum_{i \in I_r} s_i x_i + L_r \leq C \quad \forall r \in \mathcal{R}, \quad (2)$$

where C is the cache capacity limit.

This formulation is termed *pessimistic* because it assumes a worst-case space requirement. By forcing the cache to hold all currently active intervals *and* the entire length of the incoming request simultaneously at the start of the request, we ignore the fact that in a real execution, some blocks could be evicted mid-request, or the request’s tokens are consumed/generated sequentially rather than taking up the full L_r capacity all at once. While providing a strict lower bound on what can be safely cached without out-of-memory errors, performing the capacity check exclusively before serving the first block ensures sufficient space is allocated up front and significantly reduces the number of constraints in the ILP solver.

4.2 Efficient Offline Algorithm: Belady Compute

Despite the optimization largely reducing the number of constraints, the ILP problem is still computationally expensive. To address this, we introduce Belady Compute, an efficient offline algorithm that approximates the optimal solution.

The Belady Compute algorithm is inspired by the same fundamental idea as Belady’s algorithm [4, 11] (or Belady Size in variable-size caching), which evicts the item that will be needed furthest in the future. To account for the heterogeneous computation cost of prefix cache blocks, Belady Compute adapts this metric by normalizing the future distance by the block’s computation cost.

Specifically, when a cache eviction is required, Belady Compute selects the block that maximizes the following eviction metric:

$$\text{Eviction Metric} = \frac{\text{Time to next access}}{\text{Computation cost}}. \quad (3)$$

Here, the *time to next access* is measured discretely in terms of the number of incoming KV cache block requests before the block is accessed again, and the *computation cost* is the computation overhead that would be incurred if the block were evicted and needed to be recomputed. By evicting the block with the highest value of this metric, the algorithm prioritizes retaining blocks that will be reused soon and those that are highly expensive to recompute.

4.3 CAFRA: Cache-Aware Frequency-Recency Algorithm

We have also designed an online algorithm, Cache-Aware Frequency-Recency Algorithm (CAFRA), which prioritizes simplicity and low implementation overhead. Unlike complex predictive policies, CAFRA is a straightforward online policy that requires minimal state—only a per-block access counter and a last-access timestamp.

To balance frequency, recency, and computation costs without manual parameter tuning, CAFRA evaluates blocks using a single unified metric. When an eviction is triggered, CAFRA selects the block b that minimizes:

$$\text{Eviction Metric}(b) = \frac{\text{Frequency}'(b) \cdot C(b)}{\text{Recency}(b)}, \quad (4)$$

where $\text{Frequency}'(b)$ is the adjusted access frequency, $\text{Recency}(b)$ is the time since the last access (e.g., measured in request counts), and $C(b)$ is the computation cost.

To protect the cache from being polluted by blocks that are accessed only once (i.e., “one-hit wonders”), CAFRA implements a *quick demotion* mechanism. If a block has only been accessed once ($\text{Frequency}(b) = 1$), its priority is significantly reduced by applying a penalty factor (e.g., $\text{Frequency}'(b) = 0.1$). For all other blocks, the adjusted frequency is equal to the actual access count ($\text{Frequency}'(b) = \text{Frequency}(b)$).

This simple ratio inherently implements an aging mechanism: a historically popular block’s priority naturally decays as it remains idle. By scaling the metric by $C(b)$ and applying the quick demotion penalty, CAFRA efficiently retains expensive-to-compute blocks while allowing active, recently accessed sequences to replace stale or transient ones. This approach provides a practical, low-overhead solution for computation-aware prefix caching.

5 System Design

In this section, we present CHAFRA (Cache-Hole-Aware Frequency-Recency Algorithm), a system that efficiently manages prefix-matched KV caches while addressing the performance impact of fragmentation. We first analyze the impact of “holes” in the KV cache (§5.1) and then detail our hole-aware algorithm design (§5.2). Finally, we describe the implementation of CHAFRA in vLLM.

5.1 Hole Analysis: Minimal Acceptable Overhead for Each Hole

As discussed in Section 2.4, the logical structure of the prefix tree is physically decoupled from cache eviction. Because we evict the KV cache on a physical block basis rather than at the granularity of entire sequences or tree nodes, some intermediate blocks within a cached prefix may be evicted while surrounding blocks remain in memory. This block-level eviction

natively creates “holes” in the cached sequence. Furthermore, for sample-based eviction algorithms, the probabilistic nature of sampling can exacerbate this effect, potentially causing hundreds of holes within a single sequence.

Small holes make the prefill less efficient. This is primarily because the scheduling overhead is small compared to the small prefill time that each hole results in. Therefore, a hole-aware eviction and allocation algorithm is strictly necessary. When limited to a few large holes, caching fragmented blocks rescues massive computational cost. However, if an unmanaged or sample-based eviction policy randomly scatters hundreds of microscopic holes across the context, the accumulated prefill time for these microscopic holes will completely negate any attention scaling savings. Thus, a successful hole-aware design must maximize cache reuse while explicitly bounding the total number of fragmentation holes.

5.2 Hole-aware Algorithm Design

To implement a hole-aware caching strategy that bounds fragmentation while providing efficient eviction, CHAFRA builds a radix tree structure in memory. This allows us to track the physical blocks for cached prefixes and sample on the radix tree nodes directly rather than on individual blocks. The core principle of our approach is to try evicting continuous blocks within the same node. By prioritizing the eviction of contiguous blocks instead of randomly scattering victims, we naturally avoid severe fragmentation and minimize the number of holes created in the cache.

Every time eviction is triggered, we sample a subset of candidate nodes. We pick one canonical block from each node to calculate its eviction score (e.g., based on the frequency-aging metric from our CAFRA algorithm). Once a victim node is selected based on these scores, we attempt to evict blocks sequentially from the beginning of the node to its end. Finally, if all blocks in the node are evicted and it becomes a leaf node, we prune it from the radix tree.

The high-level eviction protocol operates in a tiered priority to minimize the disruptive impact of holes. As described in Algorithm 1, we systematically search for evictable blocks through the following ordered stages, accumulating freed blocks (E) until the requested limit (N) is satisfied:

1. **Unhashed blocks queue (Q_{unhashed}):** We first pop from the unhashed blocks queue to evict blocks that do not have hashes, which include non-full blocks and free blocks.
2. **Blocks not in tree ($Q_{\text{not-in-tree}}$):** Next, we pop blocks that are fully formed but not linked into the tree (e.g., blocks created during speculative decoding that were ultimately not accepted).
3. **Free radix tree nodes (S_{Free}):** Then we sample a candidate subset of k_{sample} free radix tree nodes ($\mathcal{V}_{\text{sample}}$) from

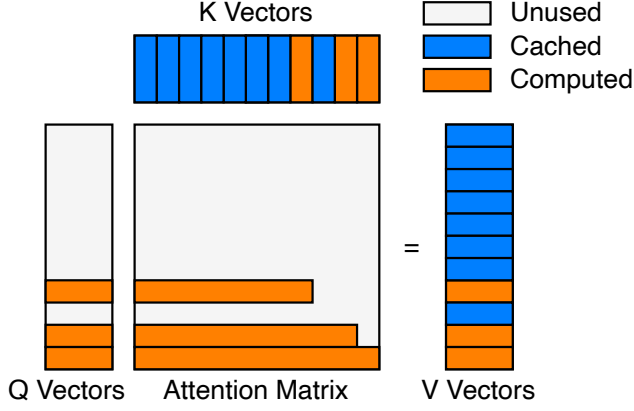


Figure 4: Batched hole-filling implementation. Instead of sequential processing, CHAFRA aggregates all missing tokens to compute QKV projections and attention in parallel, minimizing scheduling overhead.

the free node list S_{Free} that are not currently referenced by any running requests. We model an eviction score ($\text{SCORE}(v)$, e.g., representing hotness) such that the lowest score is better for eviction, and select the candidate v^* with the lowest score, sequentially evicting blocks within it.

4. **In-use radix tree nodes (S_{InUse}):** As a last resort, we sample and evict from the in-use radix tree node list S_{InUse} similarly. While most blocks are referenced by running requests (i.e., unevictable), unreferenced portions of them may sometimes be evicted.

5.3 Implementation

We implemented CHAFRA in vLLM [9]. Because the original codebase assumes prefix matching is strictly contiguous and prefix-only, we introduced several abstraction layers to facilitate segmented prefix matching.

When a request arrives, the system performs a maximal segmented match against the radix tree to identify all reusable prefix segments. This matching process produces a list of `KVCacheBlockSegment` objects, each representing a sequence of contiguous blocks found in the cache. Once identified, these segments are immediately referenced to prevent eviction during the execution cycle.

To handle the gaps between these hits, CHAFRA implements an optimized hole-filling mechanism. Rather than filling holes sequentially in multiple rounds, we exploit the lack of dependencies between disparate missing segments when the surrounding context is already cached. As shown in Figure 4, the system calculates the QKV projections for all tokens belonging to any hole simultaneously. It then computes the parts of the attention matrix corresponding to these holes in a single batched operation. By processing all holes together, we

Algorithm 1 Hole-aware Eviction Logic

```

1: procedure EVICTBLOCKS( $N$ )
2:    $E \leftarrow 0$ 
3:    $\triangleright$  1. Try to get free blocks from unhashed queue
4:    $E \leftarrow E + \text{EVICTFROMQUEUE}(Q_{\text{unhashed}}, N - E)$ 
5:   if  $E = N$  then return  $N$ 
6:   end if
7:    $\triangleright$  2. Try to get free blocks from blocks not in tree
8:    $E \leftarrow E + \text{EVICTFROMQUEUE}(Q_{\text{not-in-tree}}, N - E)$ 
9:   if  $E = N$  then return  $N$ 
10:  end if
11:   $\triangleright$  3. Try to get free blocks from free radix tree nodes
12:   $E \leftarrow E + \text{EVICTFROMNODES}(S_{\text{Free}}, N - E)$ 
13:  if  $E = N$  then return  $N$ 
14:  end if
15:   $\triangleright$  4. Try to get free blocks from in-use radix tree nodes
16:   $E \leftarrow E + \text{EVICTFROMNODES}(S_{\text{InUse}}, N - E)$ 
17:  return  $E$ 
18: end procedure

19: function EVICTFROMQUEUE( $Q, M$ )
20:    $k \leftarrow \min(M, |Q|)$ 
21:    $\text{POP}(Q, k)$ 
22:   return  $k$ 
23: end function

24: function EVICTFROMNODES( $S, M$ )
25:    $E \leftarrow 0$ 
26:   while  $E < M$  and  $S \neq \emptyset$  do
27:      $\mathcal{V}_{\text{sample}} \leftarrow \text{SAMPLENODES}(S, k_{\text{sample}})$ 
28:     if  $\mathcal{V}_{\text{sample}} = \emptyset$  then break
29:     end if
30:      $v^* \leftarrow \text{argmin}_{v \in \mathcal{V}_{\text{sample}}} \text{SCORE}(v)$ 
31:      $E \leftarrow E + \text{EVICTBLOCKSFROMNODE}(v^*, M - E)$ 
32:   end while
33:   return  $E$ 
34: end function

35: function EVICTBLOCKSFROMNODE( $v, M$ )
36:    $E \leftarrow 0$ 
37:   for  $b \in v$  do
38:     if  $E = M$  then break
39:     end if
40:     if  $b$  in memory and  $b.\text{ref\_cnt} = 0$  then
41:        $\text{EVICT}(b); E \leftarrow E + 1$ 
42:     end if
43:   end for
44:   if  $v$  is leaf node and  $v$  is empty then
45:      $\text{PRUNE}(v)$ 
46:   end if
47:   return  $E$ 
48: end function

```

significantly reduce scheduling overhead and kernel launch latency, ensuring that fragmented cache hits remain computationally efficient. In the event of preemption, the system safely unreferences both the integrated sequences and any pending hit segments that have yet to be processed, ensuring memory is correctly reclaimed.

Additionally, we developed a new abstraction layer for block management that replaces the original design, which was strictly tied to an LRU policy. This framework enables granular tracking of block access patterns and maintains a radix tree to support diverse eviction policies. Using this architecture, we implemented Belady-Compute, and CAFRA with the hole-aware algorithm design detailed in §5.2.

6 Evaluation

6.1 Evaluation Setup

Hardware and Software Environment. All experiments are conducted on a server equipped with an NVIDIA RTX PRO 6000 Blackwell GPU and an AMD Ryzen Threadripper PRO 7975WX 32-Core CPU. We implement CHAFRA on top of the vLLM [9] serving engine. Our implementation replaces the default LRU-based prefix caching manager with the hole-aware management system and CAFRA algorithm described in Section 5.3.

Model Configuration. We use the Qwen3-Coder-30B-A3B-Instruct model [17] for all evaluations. This model follows a Mixture-of-Experts (MoE) architecture, which exhibits significant attention pressure at long context lengths. The model is served using 16-bit precision and is configured with a KV cache block size of 16 tokens.

Workloads and Traces. To evaluate CHAFRA under realistic serving conditions, we utilize four production traces from the Qwen-Bailian usage dataset [1]. These traces represent diverse real-world usage patterns including multi-turn conversations, code assistant tasks, and long-context retrieval queries. For each trace, we replay the requests and measure the Time-to-First-Token (TTFT), decoding throughput, and cache hit ratios.

6.2 Cache Analysis

We evaluate the computational efficiency of CHAFRA across the four production traces described in Section 6.1. Figure 5 shows the cumulative computation saved (normalized to the total prefill FLOPs) as the cache capacity increases for each workload.

Performance of Eviction Policies. Across all traces, the standard LRU policy exhibits the lowest computational savings. This is because LRU treats all cache blocks as having equal value, frequently evicting deep blocks that are extremely expensive to recompute simply because they have not been accessed recently. In contrast, our proposed online algorithm,

CAFRA, significantly outperforms LRU by explicitly protecting these high-value, deep blocks. By incorporating the recomputation cost $C(b)$ into the eviction metric, CAFRA ensures that the most computationally intensive portions of the prefix tree remain in the cache.

Near-Optimality of Heuristics. A key observation from Figure 5 is that the Belady-Compute heuristic serves as an excellent approximation of the Pessimistic Offline Optimal (ILP) policy. In all four workloads, the performance curve for Belady-Compute is nearly indistinguishable from the ILP solution. This result validates our strategy of normalizing the time-to-next-access by the computation cost, demonstrating that it effectively captures the optimal trade-off between temporal locality and recomputation overhead.

System Improvements. CHAFRA achieves the most significant gains in the coding trace (Figure 5d), where prefix trees are deeper and attention pressure is more acute. In contrast, for traces with less pronounced prefix reuse or shorter shared contexts, such as the reasoning trace (Figure 5c), the benefits of computation-aware eviction are more subtle, though CHAFRA still maintains a lead over the LRU baseline. These results confirm that a computation-aware and hole-aware design is most effective in workloads with high attention pressure, as it shifts the system’s optimization goal from simple cache hits to overall computational efficiency.

6.3 Latency

We further evaluate the impact of CHAFRA on the Time-to-First-Token (TTFT) latency. Figure 6 illustrates the average TTFT across different cache capacities for the four traces.

The latency results strongly correlate with the computational savings observed in Section 6.2. For workloads with high attention pressure, such as the coding assistant trace, CHAFRA reduces the average TTFT by ensuring that the most expensive-to-recompute blocks remain cached. This optimization is critical for long-context serving, as it prevents the quadratic recomputation penalty that occurs when deep prefix blocks are evicted. Furthermore, the consistent performance of CAFRA across varying cache sizes demonstrates its robustness in maintaining low latency even under memory pressure.

Interestingly, we observe in certain traces that an extremely limited cache capacity (e.g., 4K blocks) may report a lower average TTFT than a slightly larger capacity (e.g., 6K blocks). This counter-intuitive trend is primarily due to request filtering: at very low capacities, the system is forced to skip or reject requests whose context lengths exceed the available physical memory. Since these skipped requests are precisely those that would incur the highest prefill latency, the resulting average TTFT only reflects shorter, less expensive queries. As the cache size increases (e.g., to 6K), the system begins to accommodate these longer requests, which naturally increases the overall average latency despite the larger cache providing

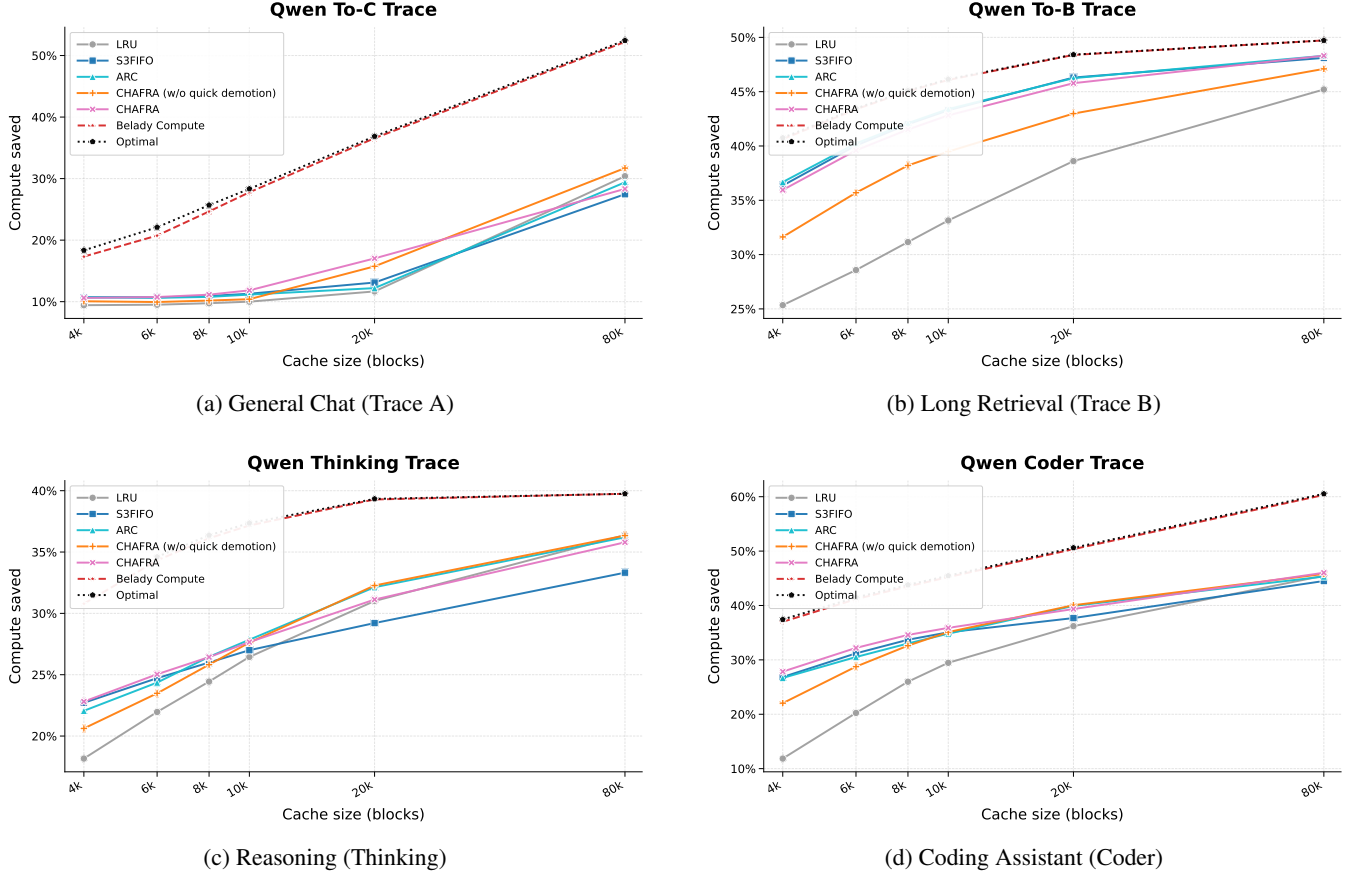


Figure 5: Cumulative computational savings across different cache sizes for four Qwen-Bailian traces. CHAFRA (CAFRA) consistently outperforms LRU and closely follows the Belady-Compute heuristic and the Offline Optimal (ILP).

more reuse opportunities.

7 Conclusion

In this project, we have presented CHAFRA (Cache-Hole-Aware Frequency-Recency Algorithm), a system designed to address the computational bottlenecks of prefix caching in long-context LLM serving. By recognizing that the re-computation cost of a KV cache block scales quadratically with its position in the sequence, we introduced the CAFRA (Cache-Aware Frequency-Recency Algorithm) algorithm. Unlike traditional eviction policies, CAFRA explicitly prioritizes the retention of deep, high-cost blocks, thereby minimizing the Time-to-First-Token (TTFT) and overall computational overhead.

Furthermore, we developed a hole-aware management system that handles the fragmentation inherent in block-level cache eviction. By implementing a batched hole-filling mechanism, CHAFRA enables the efficient reuse of non-contiguous cached segments without incurring the scheduling penalties typical of sequential recomputation. Our evaluation across diverse production traces confirms that CHAFRA significantly

outperforms standard LRU-based policies and closely approximates the performance of offline optimal strategies. As LLM context windows continue to expand, the transition from simple cache-hit optimization to computation-aware caching will be essential for maintaining high-performance serving infrastructures.

References

- [1] Alibaba Cloud. Qwen-bailian usage traces (anonymous), 2024.
- [2] Malika Aubakirova, Alex Atallah, Chris Clark, Justin Summerville, and Anjney Midha. State of ai: An empirical 100 trillion token study with openrouter. *arXiv preprint arXiv:2601.10088*, 2026.
- [3] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the asso-*

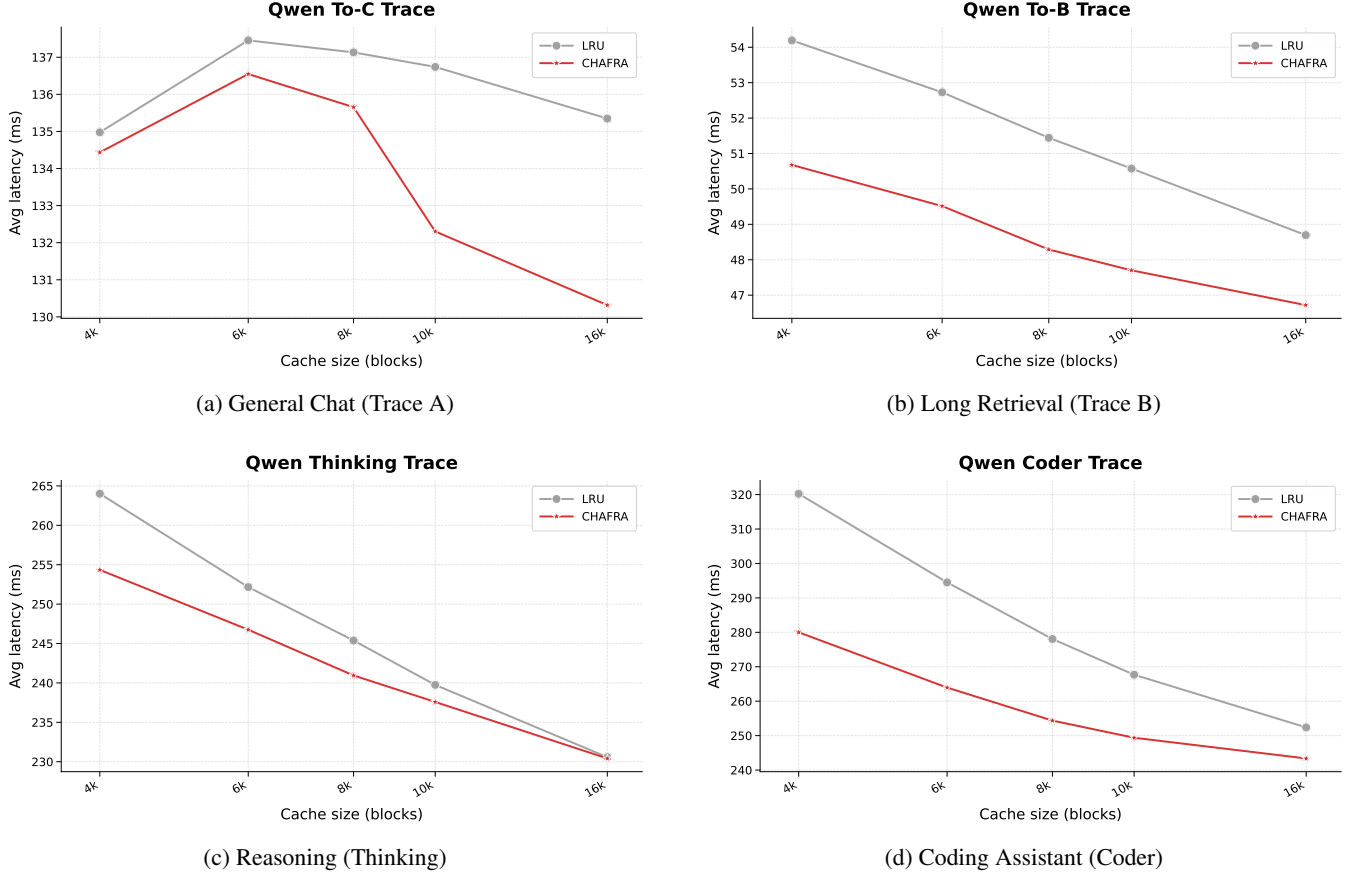


Figure 6: Average Time-to-First-Token (TTFT) for different eviction policies. By protecting deep prefix blocks, CHAFRA significantly reduces the prefill latency compared to standard LRU.

ciation for computational linguistics (volume 1: Long papers), pages 3119–3137, 2024.

- [4] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems journal*, 5(2):78–101, 1966.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [6] Character.AI. Optimizing AI inference at Character.AI, 2024.
- [7] Philip Hayes. Accelerate multi-turn LLM workloads on OpenShift AI with llm-d intelligent routing, 2026.
- [8] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- [10] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paran-jape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.
- [11] Richard L. Mattson, Jan Gecsei, Donald R. Slutz, and Irving L. Traiger. Evaluation techniques for storage hierarchies. *IBM Systems journal*, 9(2):78–117, 1970.
- [12] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium*

sium on user interface software and technology, pages 1–22, 2023.

- [13] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of the Sixth Conference on Machine Learning and Systems, MLSys 2023*, 2023.
- [14] Purav. Why Manus agents don’t break the cache and yours probably do, 2025.
- [15] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX conference on file and storage technologies (FAST 25)*, pages 155–170, 2025.
- [16] Sheng Shen, Le Hou, Yanqi Zhou, Nan Du, Shayne Longpre, Jason Wei, Hyung Won Chung, Barret Zoph, William Fedus, Xinyun Chen, et al. Mixture-of-experts meets instruction tuning: A winning combination for large language models. *arXiv preprint arXiv:2305.14705*, 2023.
- [17] Qwen Team. Qwen3 technical report, 2025.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [19] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.
- [20] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. SGLang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- [21] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35:7103–7114, 2022.